IB Mathematics Analysis and Approaches HL
Extended Essay

From Wheels to Legs: Exploring the Mobility of a Wheeled-Legged Robot with DLQR Control Techniques

How Does the Weight Matrix Q Affect the Mobility of a Wheeled-Legged Robot Using Discrete Linear Quadratic Regulator (DLQR) Control?

Word Count: 3991

Contents

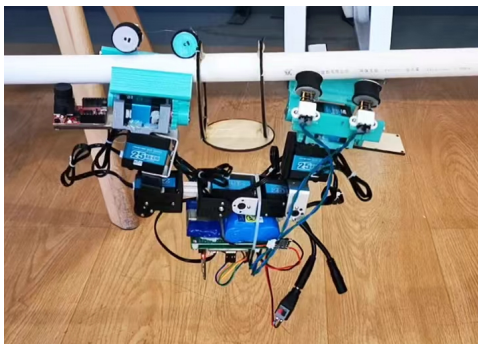
Chapter 1

Introduction

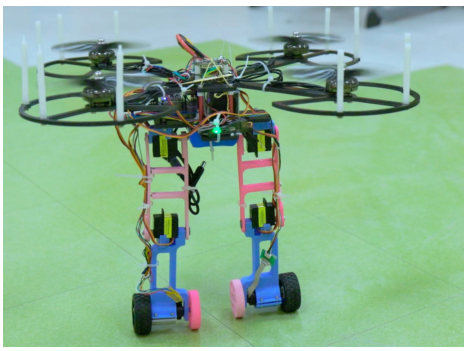
1.1 Background

Robotics has become a prominent topic recently. As an interdisciplinary subject, robotics includes wisdom from various fields, including structural engineering, hardware design, control theories, and so on. However, the recent surge in the robotics industry and research can mainly be attributed to the advancement in the control algorithm, which mathematically enables the transformation of complex systems into manageable entities. Humanoid robots, for example, walk bipedally not due to motor advancements, but because of the control algorithms behind them.

During my personal experience of building robots for more than 4 years, I have been continually captivated by the intricate algorithms propelling motors into dynamic action. As a beginner, I focus on the structure and electric circuits, writing the 'if then' statements in the codes. Later on, I realized that a more complex algorithm must be applied to dictate each motor's behavior as I delved into constructing a wheeled-legged quadcopter. When designing an algorithm for the robot in figure 1.1(b), I applied the famous



(a) Early Robot the Author Constructed as a Beginner



(b) More Advanced Robot the Author Recently Designed

Fig. 1.1: Robots Built by the Author

Proportion-Integral-Derivative (PID) algorithm [1]. PID algorithm can control a value at its prescribed state. For example, it can be used in a kettle to maintain the water temperature at 50°C. Proportion means adjusting the amount of energy supplied to the kettle's heating element in proportion to the difference e between the desired temperature and the current temperature of the water; integral indicates changing the energy inputted in proportion to the sum of error e overtime; derivative represents altering the input in proportion to the change in error e.

In this way, the PID algorithm can stabilize a system to the desired value and possesses some robustness because of the "I" and "D" terms. However, since the quantity and unit of input and output are different, this algorithm requires human adjustment to add a

coefficient to each term, namely, K_p, K_i, K_d, to ensure that the input u is not too high or too low.

$$u = K_p e + K_i \int e dt + K_d \frac{de}{dt} \tag{1.1}$$

However, despite its simplicity and practicability, the PID algorithm has many flaws. First, it requires human adjustment of the coefficient, which is empirical and inaccurate. Moreover, such an algorithm can only apply to a system with one input and one output: adding outputs requires more PID loops. I found these drawbacks especially annoying when designing my wheeled-legged quad-copter because there are 6 PID loops in total, suggesting that I must empirically adjust 18 coefficients. Thus, I began to ponder if there exists a better algorithm that doesn't require human force and meanwhile adapts to systems with multiple outputs and inputs.

1.2 Research Question

The linear quadratic regulator (LQR) [2] is a common approach to this problem. LQR can be interpreted as a model that not only helps people to solve the K_p, K_i, K_d coefficients mathematically but also allows multiple outputs and inputs, which the essay will discussed later in this essay.

The research question of this study is to explore how to build an LQR model that can effectively control a wheeled-legged robot and enhance its mobility.

1.3 Research Objectives

This research aims to rigorously derive the LQR model from scratch and analyze how it can be used to improve the control and mobility of a wheeled-legged robot.

After explaining some preliminaries, this essay will introduce and apply LQR to balance a cart-pole system(inverted pendulum) [3]. Then, discrete LQR (DLQR) will be introduced. Finally, the author will apply DLQR to a wheeled-legged robot and compare its mobility under different weight matrices.

Chapter 2

Mathematical Preliminaries

2.1 Linear Algebra

2.1.1 Basic Concepts in Matrices

When describing concepts and theorems in control theory and linear systems, matrices are usually applied to simplify these equations. This section will introduce some fundamental concepts in linear algebra to assist simplicity in the later discussion.

Arranging $m \cdot n$ scalars in the form of a table with m rows and n columns, an $m \cdot n$ matrix, also known as a matrix with order $m \cdot n$, is defined in this notation:

$$\mathbf{A_{m \cdot n}} = \begin{pmatrix} a_{11} & a_{21} & \cdots & a_{m1} \\ a_{12} & a_{22} & \cdots & a_{m2} \\ \vdots & \vdots & & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{mn} \end{pmatrix} \tag{2.1.1}$$

The transpose of matrix $\mathbf{A}$, denoted by $\mathbf{A}^T$, is defined by switching its rows into columns and columns into rows.

$$a_{ij}^T = a_{ji} \text{ for every } 1 \leq i \leq m, 1 \leq j \leq n \tag{2.1.2}$$

2.1.2 Basic Operations in Matrices

Two matrices are considered equal to each other only if they have the same order and elements in corresponding positions:

$$\mathbf{A} = \mathbf{B} \Leftrightarrow a_{ij} = b_{ij} \text{ for every } 1 \leq i \leq m, 1 \leq j \leq n \tag{2.1.3}$$

The addition and subtraction of the matrices are the addition and subtraction of each corresponding element, where the matrices have the same order. If matrix $\mathbf{C}$ is the addition of matrices $\mathbf{A}$ and $\mathbf{B}$, its trait can be expressed as

$$c_{ij} = a_{ij} + b_{ij} \text{ for every } 1 \leq i \leq m, 1 \leq j \leq n \tag{2.1.4}$$

The scalar multiplication between a matrix $\mathbf{A}$ and a scalar k can be easily defined by extending equation 2.1.4. Suppose matrix D is the product between a matrix $\mathbf{A}$ and a scalar k. Matrix $\mathbf{D}$ will have the same order as matrix $\mathbf{A}$, and its elements are defined as

$$d_{ij} = k \cdot a_{ij} \text{ for every } 1 \leq i \leq m, 1 \leq j \tag{2.1.5}$$

The matrix multiplication between matrices is more complicated. Suppose matrix $\mathbf{P}$ has an order of $m \cdot s$, and matrix $\mathbf{Q}$ has an order of $s \cdot n$. The product between matrices $\mathbf{P}$ and $\mathbf{Q}$ is defined as a new matrix $\mathbf{G}$, with the trait of

$$\mathbf{G}_{ij} = \sum_{k=1}^{s} \mathbf{P}_{ik} \cdot \mathbf{Q}_{kj} \text{ for every } 1 \leq i \leq m, 1 \leq j \tag{2.1.6}$$

For instance, if

$$\mathbf{P} = \begin{pmatrix} 3 & 4 \\ -1 & 2 \end{pmatrix}, \mathbf{Q} = \begin{pmatrix} 1 & 0 & 2 \\ 0 & -1 & 3 \end{pmatrix} \tag{2.1.7}$$

The product of $\mathbf{P}$ and $\mathbf{Q}$ can be expressed as

$$\mathbf{PQ} = \begin{pmatrix} 3 \times 1 + 4 \times 0 & 3 \times 0 + 4 \times (-1) & 3 \times 2 + 4 \times 3 \\ (-1) \times 1 + 2 \times 0 & (-1) \times 0 + 2 \times (-1) & (-1) \times 2 + 2 \times 3 \end{pmatrix} \tag{2.1.8}$$

$$= \begin{pmatrix} 3 & -4 & 18 \\ -1 & -2 & 4 \end{pmatrix} \tag{2.1.9}$$

If the elements of a matrix are functions of time t, denote them as $\mathbf{A}_{ij}(t)$. Such a matrix can be written as $\mathbf{A}(t)$. Its derivative with respect to time is a matrix of the same dimensions, with each element being the derivative of the corresponding element of the original matrix with respect to time, that is,

$$\frac{d}{dt}\mathbf{A} \stackrel{\text{def}}{=} \left(\frac{d\mathbf{A}_{ij}}{dt}\right)_{m \times n} \tag{2.1.10}$$

2.2 Partial Derivatives

Partial derivative is a fundamental concept in calculus, particularly useful in dealing with functions of multiple variables. Consider a function f depends on several variables, such as x and y, The partial derivative of f with respect to one of these variables, say x, is denoted by $\frac{\partial f}{\partial x}$.

Mathematically, the partial derivative of f with respect to x is defined as:

$$\frac{\partial f}{\partial x} = \lim_{\Delta x \to 0} \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x} \tag{2.2.1}$$

Similarly, the partial derivative of f with respect to y is:

$$\frac{\partial f}{\partial y} = \lim_{\Delta y \to 0} \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y} \tag{2.2.2}$$

For example, consider the function:

$$f(x, y) = x^2 y + 3xy^2 \tag{2.2.3}$$

The partial derivatives of f with respect to x and y are calculated as follows:

$$\frac{\partial f}{\partial x} = 2xy + 3y^2, \ \frac{\partial f}{\partial y} = x^2 + 6xy \tag{2.2.4}$$

These derivatives indicate how f changes as x or y varies while keeping the other variable fixed.

If there are n such variables, they can typically be represented as an n-dimensional column matrix, for example,

$$\mathbf{q} = (q_1, q_2, \cdots, q_n)^T \tag{2.2.5}$$

Consider this set of variables as the independent variables of a multivariable scalar function $a = a(q_1, q_2, \cdots, q_n)$, which can be simply written as $a(\mathbf{q})$. The partial derivative, or gradient, of this scalar function with respect to the j-th independent variable q_j is denoted as $\frac{\partial a}{\partial q_j}$ (for $j = 1, 2, \cdots, n$). The n gradients form an n-dimensional row matrix. The gradient of the scalar function $a(\mathbf{q})$ with respect to the variable matrix $\mathbf{q}$ are denoted as

$$\frac{\partial a}{\partial \mathbf{q}} \overset{\text{def}}{=} \left(\frac{\partial a}{\partial q_1}, \frac{\partial a}{\partial q_2}, \cdots, \frac{\partial a}{\partial q_n} \right) = \left(\frac{\partial a}{\partial q_j} \right)_{1 \times n} \tag{2.2.6}$$

For example, consider the function:

$$a(q_1, q_2) = q_1^2 q_2 + 3 q_1 q_2^2 \tag{2.2.7}$$

The gradient of a with respect to q_1 and q_2 are calculated as follows:

$$\frac{\partial a}{\partial q_1} = 2 q_1 q_2 + 3 q_2^2, \ \frac{\partial a}{\partial q_2} = q_1^2 + 6 q_1 q_2 \tag{2.2.8}$$

Thus, the row matrix of the gradients is:

$$\frac{\partial a}{\partial \mathbf{q}} = \left(\frac{\partial a}{\partial q_1}, \frac{\partial a}{\partial q_2} \right) = \left(2 q_1 q_2 + 3 q_2^2, q_1^2 + 6 q_1 q_2 \right) \tag{2.2.9}$$

2.3 Lagrangian Mechanics

Lagrangian mechanics [4] is a formulation of classic mechanics that can deal with complex physics situations. The concept of force, defined by Newton, helps to analyze motion and energy of objects easily. However, because Newtonian mechanics primarily uses Cartesian coordinates (x, y, and z) to describe the positions and motions of objects, it is difficult to apply it to complex situations.

Conversely, Lagrangian mechanics provides a more refined and comprehensive method. Generalized coordinates and generalized forces are introduced in place of the idea of forces. The ideas of energy and least action (the minimum principle) make it mathematically solvable.

During calculation, *Lagrangian* L is defined as $L = T - V$, where T is the total kinetic energy and V is the total potential energy. The generalized force Q_j is expressed with generalized coordinate q_j as:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_j}\right) - \frac{\partial L}{\partial q_j} = Q_j \tag{2.3.1}$$

2.4 State-Space Model

State-Space Models [5] are widely applied in control theory to present dynamic systems. Writing dx/dt in term of $\dot{x}$, it takes the form of

$$\dot{x}(t) = \mathbf{A}x(t) + \mathbf{B}u(t) \tag{2.4.1}$$
$$y(t) = \mathbf{C}x(t) + \mathbf{D}u(t) \tag{2.4.2}$$

- $t \in \mathbb{R}$ presents time
- $x(t) \in \mathbb{R}^n$ is the state vector
- $u(t) \in \mathbb{R}^m$ is the input vector
- $y(t) \in \mathbb{R}^p$ is the output vector
- $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the dynamics matrix
- $\mathbf{B} \in \mathbb{R}^{n \times m}$ is the input matrix
- $\mathbf{C} \in \mathbb{R}^{p \times n}$ is the output matrix
- $\mathbf{D} \in \mathbb{R}^{p \times m}$ is the feed-through matrix

It's important to note that "input" and "output" can mean different things depending on the context. In a state-space model, "input" refers to the forces acting on the system. In robotics, however, "input" usually refers to data from sensors, while the forces applied to the system are seen as the "output."

To exemplify the combination of Lagrangian Mechanics and State-Space Model, the example of a cart-pole system shown in figure 2.1 can be used. A bar linked with a ball mass is loosely connected to the cart. The weight of the bar is neglected, and the input of this system is the horizontal force F. The other symbols are defined in table 2.1.

Symbol	Meaning	Unit
s	Displacement of the cart from the central position	m
q	Angle between the pole and the vertical axis	rad
l	Length of the bar	m
M	Mass of the cart	kg
m	Mass of the ball	kg

Table 2.1: Symbols and their meanings in figure 2.1

The first step is to calculate the $L(s, q)$ with the total kinetic energy and the total potential energy:

$$T = \frac{1}{2}M\dot{s}^2 + \frac{1}{2}m[(\dot{s} - lcos(q)\dot{q})^2 + (lsin(q)\dot{q})^2] \tag{2.4.3}$$

$$V = mglcos(q) + \frac{h_{cart}}{2} \tag{2.4.4}$$

$$L(s, q) = T - V = \frac{1}{2}(M + m)\dot{s}^2 + \frac{1}{2}ml^2\dot{q}^2 - mlcos(q)\dot{q}\dot{s} - mglcos(q) \tag{2.4.5}$$

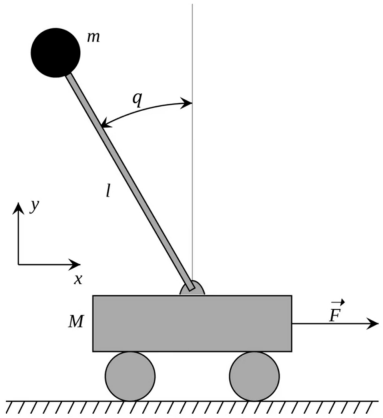


Fig. 2.1: Free body diagram of a cart-pole system, where l, q, M, m, s represent bar length, the angle between the pole and the vertical axis, the mass of the cart, the mass of the ball, and the displacement of the cart from the central position, respectively

Then, $L(s, q)$ can be put in 2.3.1 and compute the result.

$$\begin{cases} \frac{d}{dt}(\frac{\partial L}{\partial \dot{s}}) - \frac{\partial L}{\partial s} = F \\ \frac{d}{dt}(\frac{\partial L}{\partial \dot{q}}) - \frac{\partial L}{\partial q} = 0 \end{cases} \tag{2.4.6}$$

By computing the partial derivatives, it can be obtained that

$$\begin{cases} (M+m)\ddot{s} + mlsin(q)\dot{q}^2 - mlcos(q)\ddot{q} = F \\ ml^2\ddot{q} - mlcos(q)\ddot{s} - mglsin(q) = 0 \end{cases} \tag{2.4.7}$$

Computing the state-space model is a preparation of the later discussion. In the later discussion, the aim is to balance the mass at the center position by varying the input F, so this research only considers the situation near the target position. Therefore, the author linearizes the system by taking $sin(q) \approx q$ and $cos(q) \approx 1$. Also, the derivatives with higher power can be neglected. The linearized equation has the form

$$\begin{cases} (M+m)\ddot{s} - ml\ddot{q} = F \\ ml^2\ddot{q} - ml\ddot{s} - mglq = 0 \end{cases} \tag{2.4.8}$$

By solving the equation set and describing $\dot{s}, \dot{q}, \ddot{s}, \ddot{s}$ with other variables, the state-space equation is expressed as

$$\frac{d}{dt}\begin{bmatrix} s \\ q \\ \dot{s} \\ \dot{q} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & mg/M & 0 & 0 \\ 0 & (m+M)g/(Ml) & 0 & 0 \end{bmatrix}}_{\mathbf{A}} \begin{bmatrix} s \\ q \\ \dot{s} \\ \dot{q} \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ 0 \\ 1/M \\ 1/(Ml) \end{bmatrix}}_{\mathbf{B}} F \tag{2.4.9}$$

$y(t) = \mathbf{C}x(t) + \mathbf{D}u(t)$ is meaningless here because the output vector and the state vector is the same in this case. Matrices $\mathbf{A}$ and $\mathbf{B}$ are fixed once the system is defined, so the system can be fully characterized by this model.

Chapter 3

Introduction to Linear Quadratic Regulator

3.1 Full State Feedback

In this section, the cart-pole system described in equation 2.4.9 will be elaborated. The aim is to seek an autonomous mean to compute the force F that can balance the ball and the cart at the central position when the robot is under different states. The targeted state vector an be defined as

$$x_t(t) = \begin{bmatrix} s \\ q \\ \dot{s} \\ \dot{q} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{3.1.1}$$

To find the force F, understanding how a cart-pole robot works in real cases is important. In reality, values in state vectors can be obtained $(s, q, \dot{s}, \dot{q})$ with various sensors. Then, the force F can be applied with the motors linking to the wheels.

Intuitively, when the ball deviates to the left, the motor need to apply a force F pointing leftward to balance the ball to the central position. Also, the greater the deviation, the greater the force F should be. Similarly, when the cart is not at the central position, a force should be applied to move it back to the central position.

Therefore, the research assumes the force F, namely, the input u as

$$u = -\mathbf{k}x(t), \text{ where } \mathbf{k} \text{ is the gain matrix defined as } \begin{bmatrix} k_1 & k_2 & k_3 & k_4 \end{bmatrix} \tag{3.1.2}$$

This is known as full state feedback control, where each physical quantity in the state vector is multiplied by a corresponding coefficient and then summed together. In the following section, the assumption that full state feedback is the most optimal method will be proved and the values of the elements in the vector k will be calculated.

3.2 Cost Function

To prove the optimality of full state feedback, the performance of the robot should be expressed numerically, so the research sets up a cost function

$$J = \int_t^{t_1} \left(x^T \mathbf{Q} x + u^T \mathbf{R} u \right) \, dt, \tag{3.2.1}$$

$$\text{where } \mathbf{Q} \text{ and } \mathbf{R} \text{ are defined such that } \forall x \neq 0, u \neq 0, \ \exists \, x^T \mathbf{Q} x \geq 0, \ u^T \mathbf{R} u > 0. \tag{3.2.2}$$

In this function, the greater the deviation of the system from the target state $x_t(t)$ and the longer the system remains away from it, the larger the cost. Cost will not add up

anymore if the system reach its target state. The lower the cost, the better result the system performs. $\mathbf{Q}$ and $\mathbf{R}$ are diagonal matrices, also known as weight matrices, defined by the operator, and they can be altered to obtain different performances of the robot. For example, in the case of the cart-pole system, the upper setting $(\mathbf{Q}_1, \mathbf{R}_1)$ prioritizes the balance of s, and the lower setting $(\mathbf{Q}_2, \mathbf{R}_2)$ values the balance of q and the saving of power consumption. t_1 is the final time of control, and t is an arbitrary initial time. Also, the name "Linear Quadratic Regulator" comes from the process of regulating a linear system using a quadratic cost function.

$$\mathbf{Q}_1 = \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{R}_1 = \begin{bmatrix} 1 \end{bmatrix}, \mathbf{Q}_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix} \mathbf{R}_2 = \begin{bmatrix} 10 \end{bmatrix} \tag{3.2.3}$$

3.3 Minimizing Cost

The next step is to find the input u that minimize the cost given any condition. Define the minimum value of the cost-to-go at certain time and state as $J^\circ(x,t)$, which equals to

$$J^\circ(x,t) = \min_{u(t,t_1)} \left[\int_t^{t_1} (x^T \mathbf{Q} x + u^T \mathbf{R} u) dt \right] \tag{3.3.1}$$

Let t_m be an intermediate time between t and t_1 such that $t < t_m < t_1$. According to Bellman's principle of optimality, "an optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision".[6] Thus, the research deduces

$$J^\circ(x,t) = \min_{u(t,t_1)} \left[\int_t^{t_m} (x^T \mathbf{Q} x + u^T \mathbf{R} u) dt + \int_{t_m}^{t_1} (x^T \mathbf{Q} x + u^T \mathbf{R} u) dt \right] \tag{3.3.2}$$

$$= \min_{u(t,t_m)} \left[\int_t^{t_m} (x^T \mathbf{Q} x + u^T \mathbf{R} u) dt + \min_{u(t,t_1)} \left[\int_{t_m}^{t_1} (x^T \mathbf{Q} x + u^T \mathbf{R} u) dt \right] \right] \tag{3.3.3}$$

$$= \min_{u(t,t_m)} \left[\int_t^{t_m} (x^T \mathbf{Q} x + u^T \mathbf{R} u) dt + J^\circ(x(t_m), t_m) \right] \tag{3.3.4}$$

Then, to simplify the system, this research lets t_m approaches t and assume $t_m = t + \Delta t, x(t_m) = x + \Delta x$, where $\Delta t \to 0$. Through a Taylor expansion[7] on $J^\circ(x(t_m), t_m)$, it can be approximated and deduced that

$$J^\circ(x,t) = \min_{u(t,t_m)} \left[(x^T \mathbf{Q} x + u^T \mathbf{R} u)\Delta t + J^\circ(x,t) + \frac{\partial J^\circ(x,t)}{\partial x}\Delta x + \frac{\partial J^\circ(x,t)}{\partial t}\Delta t \right] \tag{3.3.5}$$

Dividing through by Δt, and canceling $J^\circ(x,t)$ on both sides, it can be deduced that

$$0 = \min_u \left[(x^T \mathbf{Q} x + u^T \mathbf{R} u)\frac{\Delta t}{\Delta t} + \frac{\partial J^\circ(x,t)}{\partial x}\frac{\Delta x}{\Delta t} + \frac{\partial J^\circ(x,t)}{\partial t}\frac{\Delta t}{\Delta t} \right] \tag{3.3.6}$$

In the real case, the cart-pole system always has an initial time of 0, so it can be deduced that $\partial J^\circ(x,t)/\partial t = 0$. Thus, this research finds that

$$0 = \min_u \left[(x^T \mathbf{Q} x + u^T \mathbf{R} u) + \frac{\partial J^\circ}{\partial x}\dot{x} \right] \tag{3.3.7}$$

$$= \min_u \left[(x^T \mathbf{Q} x + u^T \mathbf{R} u) + \frac{\partial J^\circ}{\partial x}(\mathbf{A}x + \mathbf{B}u) \right] \tag{3.3.8}$$

This research defines the term in brackets H. The goal now is to find the u that satisfies the equation. It is obvious that the cost function J° is quadratic by its definition, so the research assumes

$$J^\circ = x^T \mathbf{S} x, \text{ for some matrix } \mathbf{S} \text{ that satisfies } \mathbf{S} = \mathbf{S}^T \tag{3.3.9}$$

The gradient to x of this function is

$$\frac{\partial J^\circ}{\partial x} = 2x^T \mathbf{S} \tag{3.3.10}$$

The research finds the minimum value H mentioned in 3.3.8 by taking its partial derivative to u, deducing that

$$\frac{\partial H}{\partial u} = 2u^T \mathbf{R} + 2x^T \mathbf{S}\mathbf{B} = 0 \tag{3.3.11}$$

$$u^\circ = -\mathbf{R}^{-1}\mathbf{B}^T \mathbf{S} x = -\mathbf{K} x \tag{3.3.12}$$

To verify that this u can compute the minimum value, the research takes a second-order partial derivative:

$$\frac{\partial^2 H}{\partial u^2} = 2R \succeq 0 \tag{3.3.13}$$

Thus, H is convex, and u° can compute the minimum value. By now, this research has proven that the $u = -\mathbf{K}x$ method is the most optimal controlling strategy under the assumption of the cost function. To compute the values in matrix $\mathbf{K}$, the research needs to solve the assumed matrix $\mathbf{S}$. Putting $u = -\mathbf{K}x$ back to 3.3.8 and simplifying it, the research deduces that

$$0 = x^T (\mathbf{Q} - \mathbf{S}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T \mathbf{S} + \mathbf{S}\mathbf{A} + \mathbf{A}^T \mathbf{S}) x \tag{3.3.14}$$

Since the equation must be applied to any given x, the research finds that

$$\mathbf{Q} - \mathbf{S}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T \mathbf{S} + \mathbf{S}\mathbf{A} + \mathbf{A}^T \mathbf{S} = 0 \tag{3.3.15}$$

This is also known as the algebraic Ricatti equation[8]. This equation can be solved and obtain $\mathbf{S}$ easily through coding. Then, the gain matrix $\mathbf{K}$ can be obtained by 3.3.12.

3.4 Example Result

To test the algorithm, this research will apply this method to the cart-pole example discussed above by setting a simulation in *Python*. The parameters in the example are

```
l_bar = 2.0  # length of bar
M = 1.0  # [kg]
m = 0.3  # [kg]
g = 9.8  # [m/s^2]
Q = np.diag([1.0, 1.0, 1.0, 1.0])  # state cost matrix
R = np.diag([0.01])  # input cost matrix
```

This research uses Euler's method to simulate the state of the system:

```
def simulation(x, u):
    #A_s = np.eye(nx) + delta_t * A
    #B_s = delta_t * B
    A, B, A_s, B_s = get_model_matrix()
    x = A_s @ x + B_s @ u
    return x
```

Then, this research computes the gain matrix k by $SciPy$:

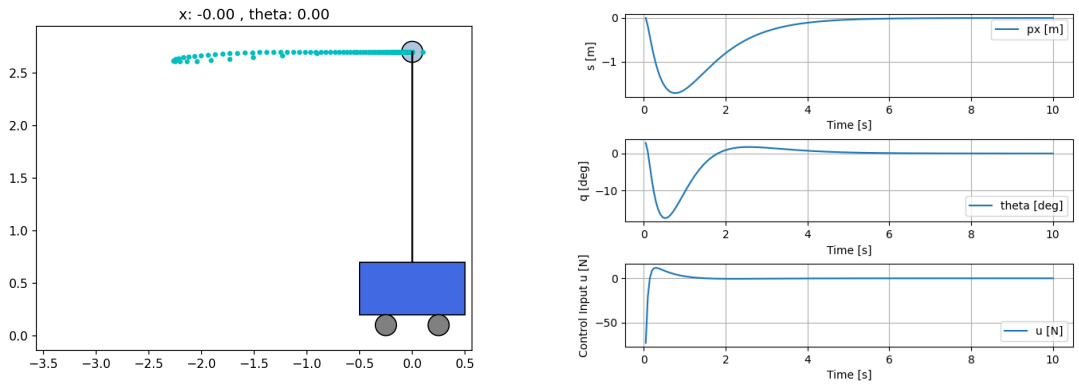
```
def lqr(A, B, Q, R):
    S = solve_continuous_are(A, B, Q, R)
    k = inv(R) @ B.T @ S
    return k, S
```

In this way, the research obtains

$$u = -Kx = -\begin{bmatrix} -10.00 & 163.80 & -20.21 & 73.20 \end{bmatrix} x \tag{3.4.1}$$

Putting it into the simulation, the research receives animations of the system starting from different initial states. For example, the initial state can be $x^T = \begin{bmatrix} 0 & 0 & 0 & 1.0 \end{bmatrix}$.

This can be interpreted as the ball mass being pushed by an external force. The result of the simulation is shown in figure 3.1, where the dotted line represents the past trace of the ball mass. From the diagram, the research concludes that as the force u varies



(a) Animation Screenshot at $t = 10s$

(b) Values of s, q, u at Different Time

Fig. 3.1: Simulation Result of the Cart-pole System controlled by LQR in the first 10 seconds

at different states, it finally manages to bring the system to the target state. At $t = 0$ seconds, when the ball mass is pushed, both s and q become negative. However, the LQR algorithm manages to compute u in such a way that q approaches 0 around $t = 2$ seconds. Meanwhile, s converges to 0 much later because changes in s can significantly affect q, causing s to approach 0 at a much slower rate.

Chapter 4

Extension to Discrete Linear Quadratic Regulator

4.1 Advantages of DLQR

The Discrete Linear Quadratic Regulator (DLQR) is an extension of LQR, designed for discrete-time systems. In practical applications, many real-world systems operate in discrete time due to the nature of digital sensors and controllers, which sample and process data at specific intervals.

DLQR is advantageous because it aligns with the operational modes of real-world digital systems. By discretizing the state-space model of a system, DLQR can optimize the performances of real-world control system.

4.2 Minimizing Cost

To discretize the system, the research changes the description of the state space model to

$$x[k+1] = \mathbf{A}x[k] + \mathbf{B}u[k] \tag{4.2.1}$$

and rewrite the cost function as

$$J = \sum_{k=k_1}^{n-1} (x[k]^T \mathbf{Q} x[k] + u[k]^T \mathbf{R} u[k]), \text{ where } Q = Q^T \succeq 0, R = R^T \succ 0 \tag{4.2.2}$$

Again, the research defines the minimum value of the cost-to-go at certain time and state as $J^\circ(x,k)$, analyzing the optimization through similar method mentioned above:

$$J^\circ(x, k-1) = \min_u \left[x^T \mathbf{Q} x + u^T \mathbf{R} u + J^\circ(\mathbf{A}x + \mathbf{B}u, k) \right] \tag{4.2.3}$$

Similar to the previous approach, the research takes

$$J^\circ(x,k) = x^T \mathbf{S}[n] x, \text{ for some matrix } \mathbf{S} \text{ that satisfies } \mathbf{S} = \mathbf{S}^T \tag{4.2.4}$$

Then, the research finds the value of u that minimizes the cost-to-go $J^\circ(x, k-1)$ in equation 4.2.3 by taking its partial derivative. By minimizing the cost-to-go from any time step $k-1$ to k, the research obtains the optimal policy for any time and state, in line with Bellman's principle of optimality discussed earlier.

$$\frac{\partial J^\circ}{\partial u} = \frac{\partial}{\partial u} \left[x^T \mathbf{Q} x + u^T \mathbf{R} u + (\mathbf{A}x + \mathbf{B}u)^T \mathbf{S}[k] (\mathbf{A}x + \mathbf{B}u) \right] \tag{4.2.5}$$

$$= 2u^T \mathbf{R} + 2\mathbf{B}^T \mathbf{S}[n] (\mathbf{A}x + \mathbf{B}u) \tag{4.2.6}$$

By letting equation 4.2.6 equal to zero, the value u that yields the minimum cost can be obtained. It can be proven that the value is a minima by applying a second order derivative similar as equation 3.3.13. The value of u is

$$u^{\circ}[k] = -(\mathbf{R} + \mathbf{B}^T\mathbf{S}[k]\mathbf{B})^{-1}\mathbf{B}^T\mathbf{S}[k]\mathbf{A}x[k] = -\mathbf{K}[k]x[k] \tag{4.2.7}$$

Again, the research proves that full-state feedback $u = -Kx$ is the most optimal control method for discrete situations. To obtain the value of K, the value of S must be computed. Inserting equation 4.2.7 to equation 4.2.3 can deduce that

$$\mathbf{S}[k-1] = \mathbf{Q} + \mathbf{A}^T\mathbf{S}[k]\mathbf{A} - (\mathbf{A}^T\mathbf{S}[k]\mathbf{B})(\mathbf{R} + \mathbf{B}^T\mathbf{S}[k]\mathbf{B})^{-1}(\mathbf{B}^T\mathbf{S}[k]\mathbf{A}) \tag{4.2.8}$$

A key difference between LQR and DLQR can be observed now. In standard LQR, a fixed matrix $\mathbf{S}$ can be computed using equation 3.3.15. In contrast, DLQR involves a time-varying matrix $\mathbf{S}[k]$. While it seems there are many possible values for $\mathbf{S}$ since $\mathbf{S}[0]$ can be any number, it will quickly converge to the same value regardless of $\mathbf{S}[0]$. This can be demonstrated through experiments. For instance, in the cart-pole system, different values for $\mathbf{S}[0]$ are tested. If $\mathbf{S}[0] = \mathbf{Q}$, the change of matrix $\mathbf{S}$ can be observed through coding.

```
def solve_DARE(A, B, Q, R, maxiter=1500, eps=0.01):
    S = Q
    for i in range(maxiter):
        Sn = A.T @ S @ A - A.T @ S @ B @ \
            inv(R + B.T @ S @ B) @ B.T @ S @ A + Q
        if (abs(Sn - S)).max() < eps:
            break
        S = Sn
        print(S)
    return Sn
```

The research finds out that $\mathbf{S}$ will converge to matrix $\mathbf{S}_c$. For other initial value $\mathbf{S}[0]$, the results are the same.

$$\mathbf{S}_c = \begin{bmatrix} 42.48 & 34.06 & -165.09 & -74.08 \\ 34.06 & 53.80 & -268.38 & -119.97 \\ -165.09 & -268.38 & 1448.49 & 635.33 \\ -74.08 & -119.97 & 635.33 & 284.25 \end{bmatrix} \tag{4.2.9}$$

Since the matrix $\mathbf{S}$ eventually converges to a steady value, the research uses that converged value right from the start. By calculating it beforehand, the code only need to calculate once instead of recalculating in every cycle.

4.3 Example Result

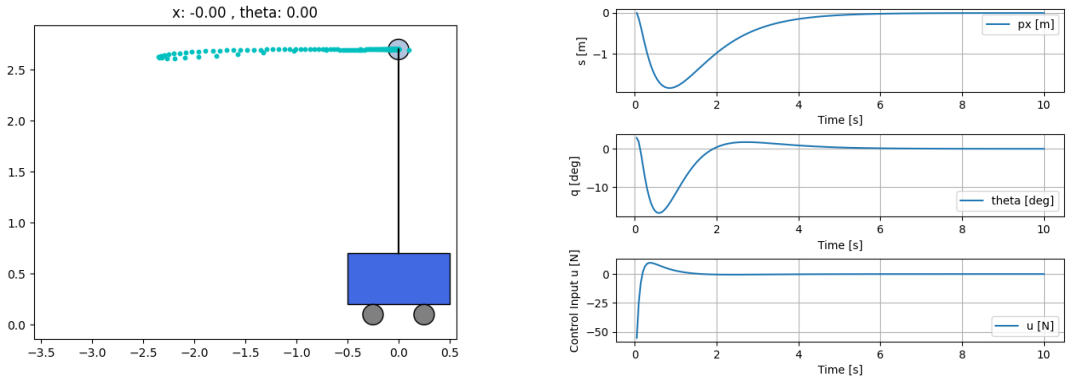
This new DLQR algorithm is applied on the same cart-pole system discussed above. The same *Python* code with LQR with some minor changes can be used again. The feedback matrix $\mathbf{K}$ is substituted with the methods discussed in equation 4.2.7.

```
def dlqr(A, B, Q, R):
    S = solve_DARE(A, B, Q, R)
    K = inv(B.T @ P @ B + R) @ (B.T @ P @ A)
    return K, S
```

The matrix K obtained is

$$u = -\mathbf{K}x = -\begin{bmatrix} -6.71 & 124.68 & -14.25 & 55.37 \end{bmatrix} \begin{bmatrix} s \\ q \\ \dot{s} \\ \dot{q} \end{bmatrix} \tag{4.3.1}$$

In this simulation, the same initial state is applied, as well as the same **Q** and **R** matrices as those used in the LQR test. The results are displayed in the figure 4.1. The results



(a) Animation Screenshot at $t = 10s$

(b) Values of s, q, u at Different Time

Fig. 4.1: Simulation Result of the Cart-pole System controlled by DLQR in the first 10 seconds

show that the cart-pole system successfully stabilizes. The state and control input curves are very similar to those from the LQR test, as the feedback matrix **K** is nearly identical. The system is too simple to reveal significant differences.

Chapter 5

Analyzing DLQR on a Wheeled-Legged Robot

5.1 Introduction to Wheeled-Legged Robot

By now, the essay has discussed how to control or regulate a system through LQR and has extended the method to DLQR. However, the key research question remains: how do different selections of the weight matrices **Q** and **R** influence system performance? In the previous cart-pole example, the performance impact of weight selections was minimal due to its single-output nature. In reality, most systems, such as robots, have multiple inputs and outputs, and the selection of weight matrices matters a lot.

The research focuses on how varying the **Q** matrix affects a system, as **R** is primarily determined by motor capabilities. To address this, the research will use the example of a wheeled-legged robot. The robot has a body and two legs, each with three degrees of freedom. Each joint is controlled by a motor, making the joint angles the inputs and the motor torques the outputs. The goal is to balance the robot in the middle position. To simplify the system, this research only focus on one side of the robot, assuming symmetry, so the situation is converted to a robot with only one leg.

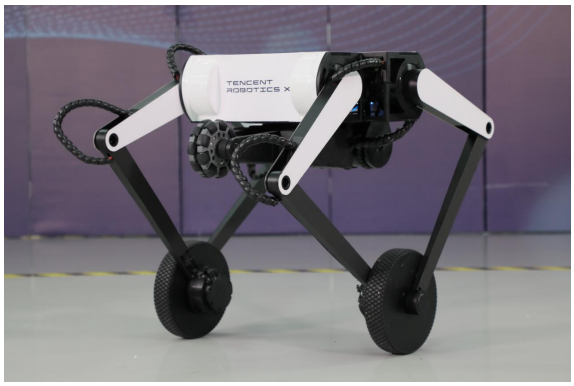


Fig. 5.1: A wheeled-legged robot from Tencent with three degrees of freedom on each leg[9]

In the later essay, a wheeled-legged robot is regulated using DLQR, and the impact of different weight matrices on system mobility is analyzed. The algorithm is tested in a simulated physical environment built with *Simulink* in *MATLAB*, based on [10].

5.1.1 Mechanical Analysis

To implement the DLQR algorithm, the research needs to build the state-space model of a simplified wheeled-legged robot with only one leg. The heavy physics part will not be fully elaborated because that is not this essay's primary focus.

To build the physics model, the research first analyzes the structure of the leg of the robot.

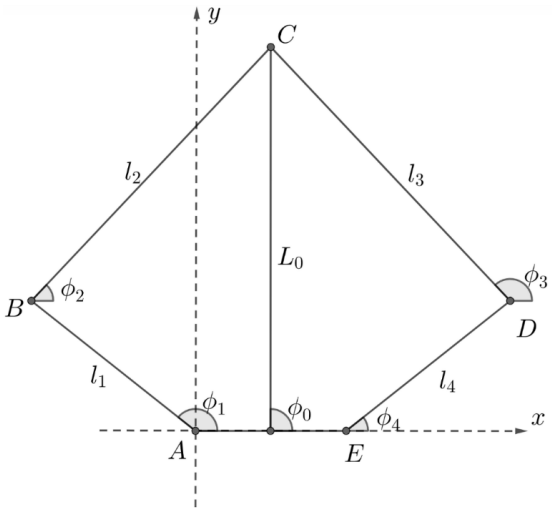


Fig. 5.2: Five-rod leg structure of a wheeled-legged robot and definition of parameters[11]

As illustrated in figure 5.2, A and E represent the central positions of two motors, while B and D indicate frictionless joints connected by bearings. C marks the central position of the motor controlling the wheel. The symbols are defined in table 5.1.

Symbol	Meaning	Unit
T_A	Torque of Motor A	$\mathrm{N}\cdot\mathrm{m}$
T_B	Torque of Motor B	$\mathrm{N}\cdot\mathrm{m}$
F_l	Component of the force at point C toward the midpoint of A and E	N
T_r	Torque perpendicular to the line from C to the midpoint of A and E	$\mathrm{N}\cdot\mathrm{m}$

Table 5.1: Symbols and their meanings in model 5.2

The torques T_A and T_E from the motors influence the motion of point C, such as its acceleration. However, calculating this is complex, and introducing such complexity into the state-space model is not desired.

Therefore, converting T_A and T_E to motion of point C prior to building the state-space model is nescessary. It is easy to see that

$$\begin{cases} x_B + l_2\cos\phi_2 = x_D + l_3\cos\phi_3 \\ y_B + l_2\sin\phi_2 = y_D + l_3\sin\phi_3 \end{cases} \tag{5.1.1}$$

Let $p = [L_0\ \phi_0]^T$, $q = [\phi_1\ \phi_4]^T$, $T = [T_A\ T_E]^T$, and $F = [F_l\ T_r]^T$. It is easy to convert q to p by rearranging equation 5.1.1. Through some physics techniques, such as Virtual Model Control[12], T can be converted to F through a matrix $\mathbf{J}$.

$$T = \mathbf{J}^T F \tag{5.1.2}$$

The value of $\mathbf{J}$ can be derived using Lagrangian mechanics. For a comprehensive explanation, refer to [12]. By transforming T into F, the physical model is reduced from a complex five-rod structure to a simplified single rod, where both its length and angle can be controlled.

With the simplified system, the state-space model of the robot is built. Let's assume L_0 is a fixed value controlled by force F_l. In reality, PID is used to control its value, so in the state-space model, the only inputs are the wheel's torque T_w and the torque T_r.

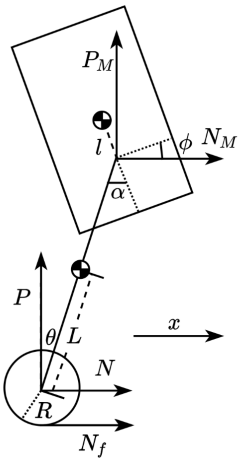


Fig. 5.3: Simplified model of a wheeled-legged robot and definition of parameters[11]

Symbol	Meaning	Unit
θ	Angle between the pendulum and vertical	rad
x	Displacement of the drive wheel	m
ϕ	Angle between the body and horizontal	rad
T_w	Output torque of the drive wheel	$\theta \cdot m$
T_r	Output torque of the suspension joint	$\alpha \cdot m$
R	Radius of the drive wheel	m
L	Distance from the pendulum center to the drive wheel axis	m
L'	Distance from the pendulum center to the body axis	m
l	Distance from the body center to its axis	m
m_w	Mass of the drive wheel	kg
m_p	Mass of the pendulum	kg
m_b	Mass of the body	kg
I_w	Moment of inertia of the drive wheel from its center	kg $\cdot$ m^2
I_p	Moment of inertia of the pendulum from the wheel's center	kg $\cdot$ m^2
I_b	Moment of inertia of the body from the body axis	kg $\cdot$ m^2

Table 5.2: Symbols and their meanings in 5.3

In figure 5.3, the circle represents the wheel, and the rectangle represents the body of the robot. The state vector x and input vector u are

$$x = \begin{bmatrix} \theta \\ \dot{\theta} \\ x \\ \dot{x} \\ \phi \\ \dot{\phi} \end{bmatrix} \quad u = \begin{bmatrix} T_w \\ T_r \end{bmatrix} \tag{5.1.3}$$

The next step is to find matrices A and B. Lagrangian mechanics is applied again to find

the answer.

$$T = \frac{1}{2}I_w\left(\frac{\dot{x}}{R}\right)^2 + \frac{1}{2}I_p\dot{\theta}^2 + \frac{1}{2}m_b\left[\dot{\theta}(L+L')\right]^2 + \frac{1}{2}I_M\phi^2 + \frac{1}{2}(m_w+m_p+m_b)\dot{x}^2 \tag{5.1.4}$$

$$U = m_p g L\cos\theta + m_b g(L+L')\cos\theta + m_b g l\cos\phi \tag{5.1.5}$$

$$L = T - U \tag{5.1.6}$$

$$\begin{cases} \dfrac{d}{dt}(\dfrac{\partial L}{\partial\dot{\theta}}) - \dfrac{\partial L}{\partial\theta} = T_w \\ \dfrac{d}{dt}(\dfrac{\partial L}{\partial\dot{\phi}}) - \dfrac{\partial L}{\partial\phi} = T_b \end{cases} \tag{5.1.7}$$

Then, through rearranging these equations in *MATLAB* and linearizing it around the balanced position, the state-space model is obtained. Let $R = 2.0\times10^{-2}$, $L = L_0/2 = 5.0\times10^{-2}$, $L_m = L_0/2 = 5.0\times10^{-2}$, $l = 0$, $m_w = 2.5\times10^{-1}$, $m_p = 2.5\times10^{-2}$, $m_b = 3.25\times10^{-1}$, $I_w = 5.027\times10^{-5}$, $I_p = 1.60\times10^{-5}$, $I_m = 3.39\times10^{-4}$, $g = 9.8$. Matrices A and B for different L_0 is obtained. For example, the value of A and B when $L_0 = 0.07$ is

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 265.033 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ -8.61259 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix},\ \mathbf{B} = \begin{bmatrix} 0 & 0 \\ -3005.59 & 1145.07 \\ 0 & 0 \\ 166.466 & -37.2104 \\ 0 & 0 \\ 0 & 2953.84 \end{bmatrix} \tag{5.1.8}$$

5.1.2 Solving DLQR and Simulation Results

With the expression of $\dot{x} = Ax + Bu$, the DLQR problem can be solved. Taking the following weight matrices as an example, the gain matrix K in $u = -Kx$ can be computed.

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 & 0 & 0 \\ 0 & 0 & 100 & 0 & 0 & 0 \\ 0 & 0 & 0 & 20 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1000 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix},\ \mathbf{R} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{5.1.9}$$

Then, through the method discussed above, the research solves the gain matrix K for this system. In *MATLAB*, the calculation is simplified as a function called $dlqr(\mathbf{A},\mathbf{B},\mathbf{Q},\mathbf{R})$. For example, when $L_0 = 0.07$, the gain matrix computed is

$$u = -\mathbf{K}x = -\begin{bmatrix} -0.5027 & -0.0782 & -0.2053 & -0.1912 & 0.7515 & 0.0292 \\ -0.0007 & 0.0001 & 0.0026 & 0.0008 & 1.9726 & 0.0723 \end{bmatrix}\begin{bmatrix} \theta \\ \dot{\theta} \\ x \\ \dot{x} \\ \phi \\ \dot{\phi} \end{bmatrix} \tag{5.1.10}$$

In this manner, the values for the outputs, which are the torques T_w and T_r, are deduced. Up to this point, the essay has discussed the simplification of the leg structure, the calculation of the state-space model, and the solution of DLQR. To enhance clarity, a flowchart has been created to visually represent the entire control process.

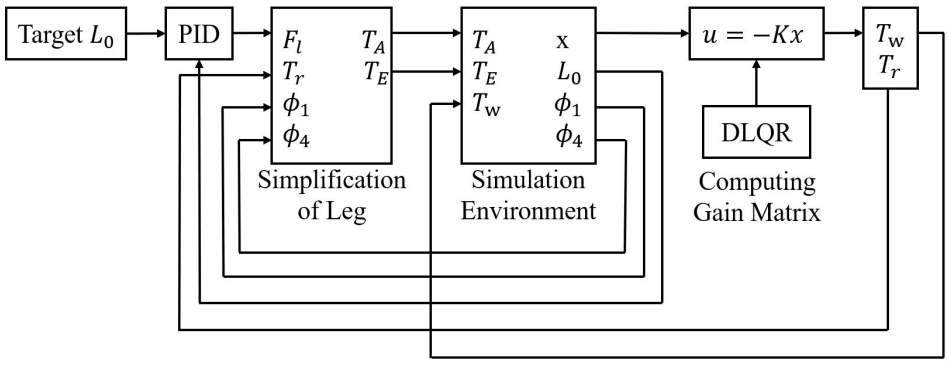
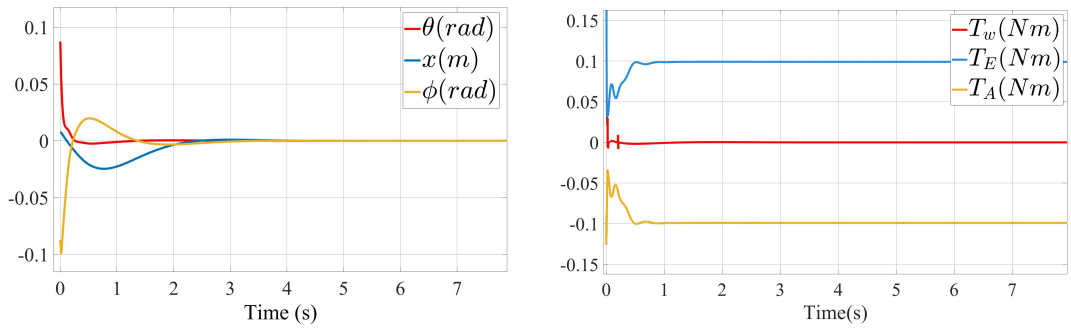


Fig. 5.4: System flow chart of a leg-wheeled robot controlled by DLQR

Let the targeted $L_0 = 0.9$, and the initial state being $\phi_1 = \pi$ and $\phi_4 = 0$. The simulation is run for 8 seconds, and the values of state matrix and outputs are shown in figure 5.5. The animation screenshots are shown in figure 5.6. In the graph, it is observed that θ, x, ϕ all

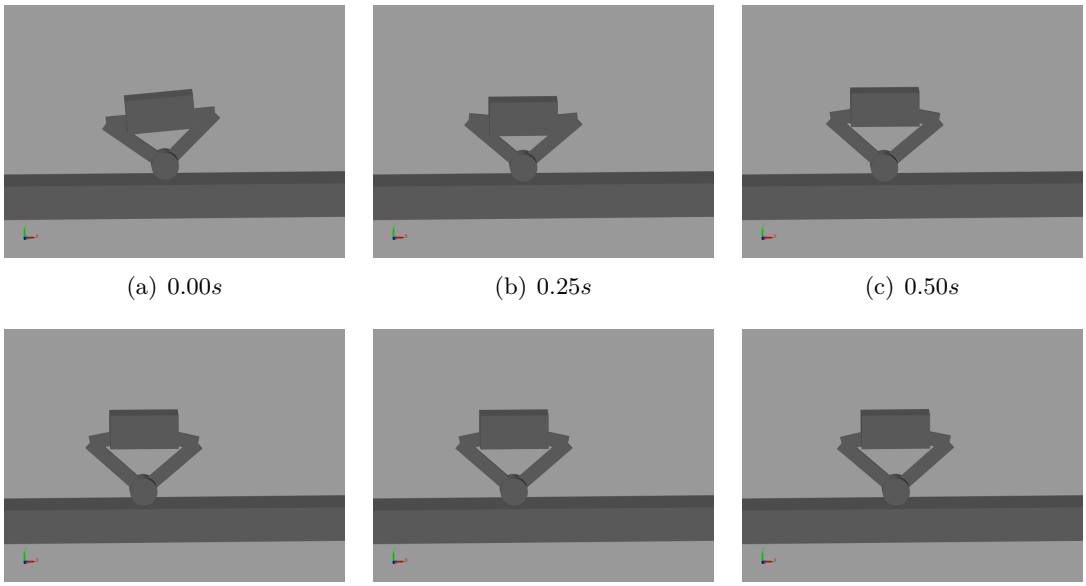


(a) θ, x, ϕ (b) T_A, T_w, T_E

Fig. 5.5: State of the robot regulated by DLQR at different time when the speed target is 0

converge to 0, showing the effectiveness of DLQR. T_w also approaches zero, and T_A, T_E also stabilize as they need to support the weight of the robot. The system stabilizes in less than 2 seconds, demonstrating DLQR's optimality.

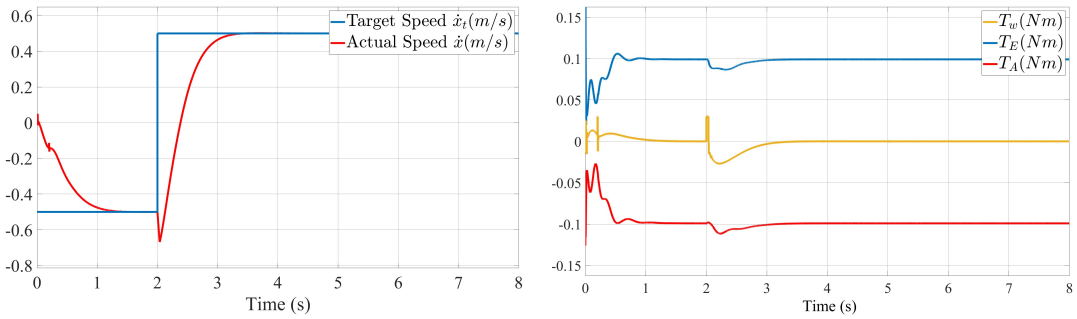
To make the robot move forward and backward using DLQR, a few small adjustments are made. First, x is set to 0 in the program, so it doesn't affect the motor outputs. Next, $\dot{x}$ is modified to $\dot{x} - \dot{x}_t$, where $\dot{x}_t$ is the target speed. This way, the motors adjust their output until the robot reaches the desired speed. The algorithm can test this by running a simulation, setting the target speed to -0.5m/s for the first 2 seconds and 0.5m/s for the next 6 seconds, and then observe the robot's speed over time. Figure 5.7 shows that the speed successfully follows the target as it changes its speed. The unusual peak around 2 seconds isn't due to a sudden speed increase. Instead, it occurs because $\dot{x}$ represents the wheel's linear speed, and the robot leans rightward to adjust its speed. This leaning posture is illustrated in Figure 5.8.



(a) $0.00s$ (b) $0.25s$ (c) $0.50s$

(d) $0.75s$ (e) $1.00s$ (f) $2.00s$

Fig. 5.6: Animation of the robot regulated by DLQR at different time when the speed target is 0



(a) $\dot{x}$ (b) T_A, T_w, T_E

Fig. 5.7: State of the robot regulated by DLQR at different time when the speed target is changing

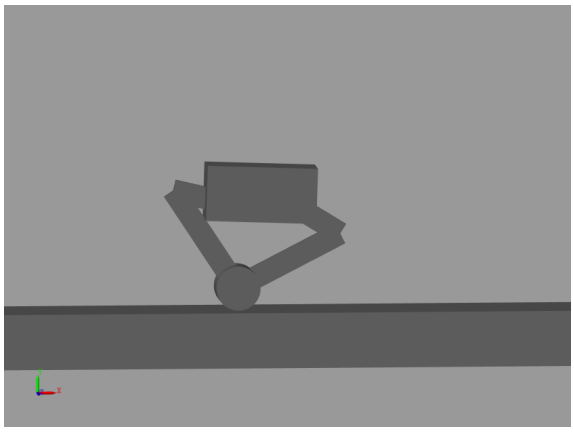


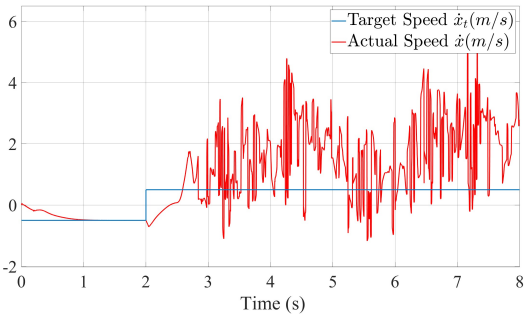
Fig. 5.8: The special leaning posture when the robot is changing its speed at $t = 2.2s$

5.1.3 Mobility Analysis with Different Weight Matrices

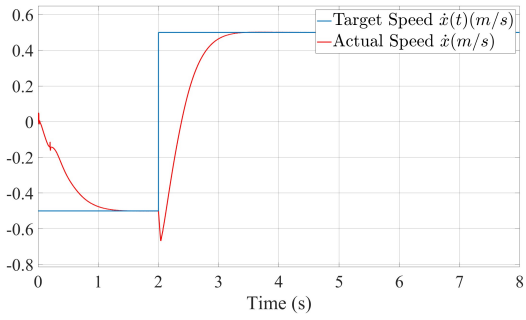
In figure 5.7, the research lets the robot change its speed at the 2nd second. This sudden speed change effectively shows the robot's mobility. Next, the research will use this method to compare the robot's mobility with different weight matrices. The faster the target speed is reached, the stronger the mobility. The experiment is divided in to 3 groups, changing the weights of $(\theta, \dot{\theta})$, $\dot{x}$, and $(\phi, \dot{\phi})$, respectively. From figure 5.9, 5.10, and 5.11, it is concluded that increases in weights of $\dot{x}, \dot{\phi}$ and decreases in weights of $\theta, \dot{\theta}, \phi$ improve the mobility and response speed of the robot.(It should be noted that ϕ and $\dot{\phi}$ have less impact on overall mobility.) However, blindly pursing higher response speed might cause lose of control in 5.9(a), 5.10(d) and obvious overshoot in 5.11(b).

The results mostly match intuition. Lower weights for θ and $\dot{\theta}$ allow the robot to lean its body more, as shown in figure 5.8, which improves turning. Higher weights for $\dot{x}$ naturally result in faster movement toward the target. The weights for ϕ and $\dot{\phi}$ have less impact on mobility, as expected, since these variables are not directly related to it.

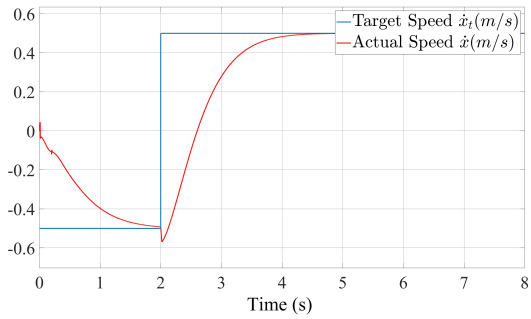
The approach used in this research, which involves the controlled variable method and observing the speed of approaching target velocity, is a simple yet effective way to qualitatively assess each variable's impact on mobility. However, its drawback is that it cannot quantitatively determine the impact, unlike more complex methods such as deducing the differential equations of speed.



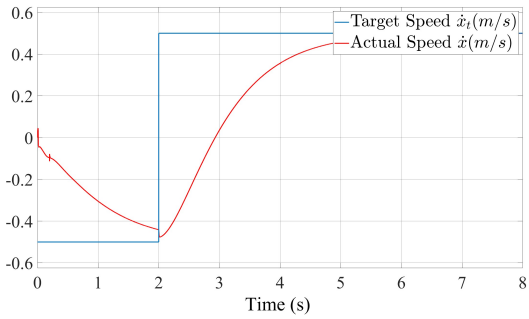
(a) $Q = diag(0.5, 5, 100, 20, 1000, 1)$ (lose control)



(b) $Q = diag(1, 10, 100, 20, 1000, 1)$

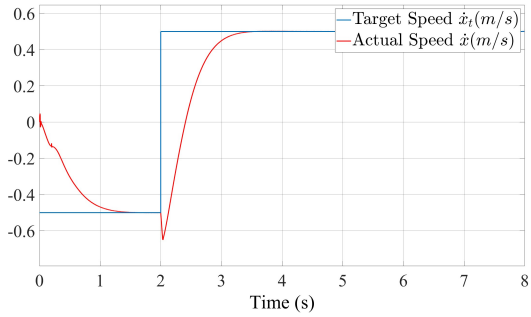


(c) $Q = diag(10, 100, 100, 20, 1000, 1)$

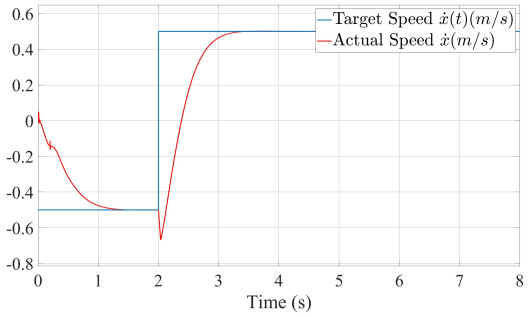


(d) $Q = diag(100, 1000, 100, 20, 1000, 1)$

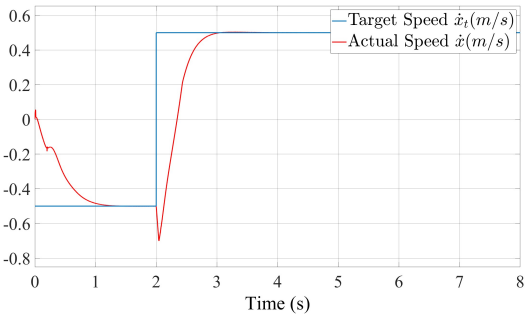
Fig. 5.9: Speed $\dot{x}$ of the robot regulated by DLQR with varying weights for $(\theta, \dot{\theta})$ in the Q matrix during changing speed targets. Lower weights for $(\theta, \dot{\theta})$ result in faster response and enhanced mobility. However, excessively low weights may lead to a loss of control.



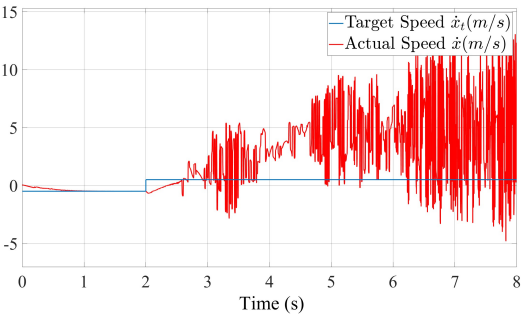
(a) $Q = diag(1, 10, 100, 5, 1000, 1)$ (lose control)



(b) $Q = diag(1, 10, 100, 20, 1000, 1)$

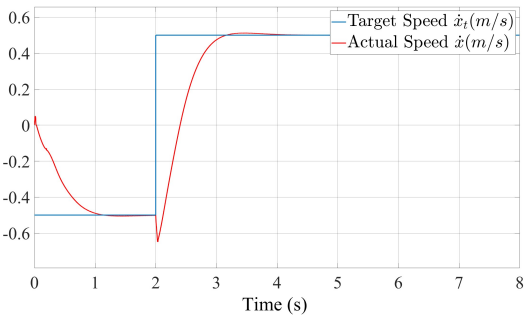


(c) $Q = diag(1, 10, 100, 60, 1000, 1)$

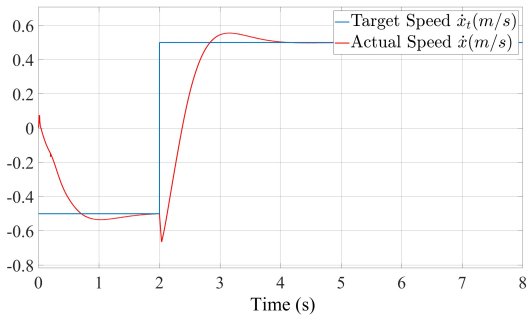


(d) $Q = diag(1, 10, 100, 100, 1000, 1)$(lose control)

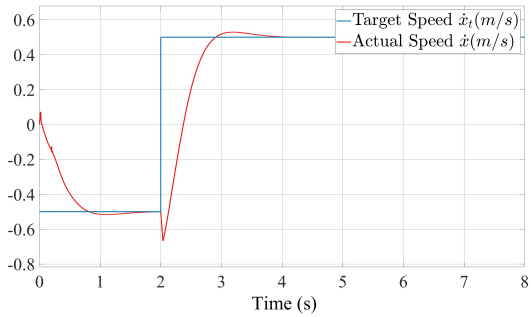
Fig. 5.10: Speed $\dot{x}$ of the robot regulated by DLQR with varying weights for $\dot{x}$ in the Q matrix during changing speed targets. Greater weights for $\dot{x}$ result in faster response and enhanced mobility. However, excessively high weights may lead to a loss of control.



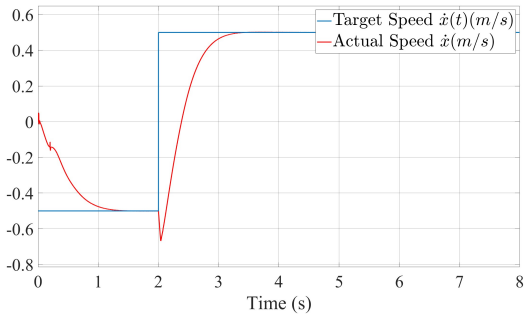
(a) $Q = diag(1, 10, 100, 20, 1, 0.1)$



(b) $Q = diag(1, 10, 100, 20, 1, 1)$



(c) $Q = diag(1, 10, 100, 20, 10, 1)$



(d) $Q = diag(1, 10, 100, 20, 1000, 1)$

Fig. 5.11: Speed $\dot{x}$ of the robot regulated by DLQR with varying weights for $(\phi, \dot{\phi})$ in the Q matrix during changing speed targets. Lower weights for ϕ result in faster response and enhanced mobility, while lower weights for $\dot{\phi}$ does the opposite.

Chapter 6

Conclusion

In conclusion, this essay explored the foundational principles of Linear Quadratic Regulator (LQR) and Discrete Linear Quadratic Regulator (DLQR) by examining strategies to minimize the cost function. Using the cart-pole system as a case study, the research illustrated how these control methods can effectively manage system dynamics.

The essay then addressed the essential research question. After analyzing the mechanics of a wheeled-legged robot, its state-space model is derived, and DLQR is implemented, simulated in *MATLAB*'s *Simulink* environment. The research then assessed the robot's mobility under different Q matrices in response to sudden changes in target speed. Its findings indicate that appropriately increasing weight of $\dot{x}$ and $\dot{\phi}$, while decreasing the weights of θ, $\dot{\theta}$ and ϕ, enhances the mobility and responsiveness of the robot.

Looking ahead, my aim is to further investigate the dynamics of wheeled-legged robots. Current research is limited to linearized models near the balanced point, which may not adequately represent scenarios outside this range. By extending the LQR and DLQR algorithms to accommodate nonlinear systems, I hope to enhance their applicability and robustness in real-world situations, paving the way for more advanced robotic control strategies.

Bibliography

[1] Michael A Johnson and Mohammad H Moradi. *PID control*. Springer, 2005.

[2] Roland Büchi. *State Space Control, LQR and Observer: step by step introduction, with Matlab examples*. Books on Demand, 2010.

[3] Lal Bahadur Prasad, Barjeev Tyagi, and Hari Om Gupta. Optimal control of nonlinear inverted pendulum dynamical system with disturbance input using pid controller & lqr. In *2011 IEEE International Conference on Control System, Computing and Engineering*, pages 540–545. IEEE, 2011.

[4] Alain J Brizard. *Introduction To Lagrangian Mechanics, An*. World Scientific Publishing Company, 2014.

[5] Jay H Lee, Manfred Morari, and Carlos E Garcia. State-space interpretation of model predictive control. *Automatica*, 30(4):707–717, 1994.

[6] Bellman equation - Wikipedia — en.wikipedia.org. https://en.wikipedia.org/wiki/Bellman_equation. [Accessed 27-05-2024].

[7] Taylor Polynomials of Functions of Two Variables — math.libretexts.org. https://math.libretexts.org/Bookshelves/Calculus/Supplemental_Modules_(Calculus)/Multivariable_Calculus/3%3A_Topics_in_Partial_Derivatives/Taylor__Polynomials_of_Functions_of_Two_Variables. [Accessed 27-05-2024].

[8] P Lancaster. *Algebraic Riccati Equations*. Oxford Science Publications/The Clarendon Press, Oxford University Press, 1995.

[9] https://www.facebook.com/48576411181. Tencent's New Wheeled Robot Flicks Its Tail To Do Backflips — spectrum.ieee.org. https://spectrum.ieee.org/tencents-new-wheeled-robot-flicks-its-tail-to-do-backflips. [Accessed 13-08-2024].

[10] GitHub - Skythinker616/foc-wheel-legged-robot: Open source materials for a novel structured legged robot, including mechanical design, electronic design, algorithm simulation, and software development. https://github.com/Skythinker616/foc-wheel-legged-robot. [Accessed 15-08-2024].

[11] https://zhuanlan.zhihu.com/p/563048952. [Accessed 13-08-2024].

[12] Xiaopeng Huang, Tao Liu, Meimei Han, and João P. Ferreira. Virtual model control for dynamic banlance of a two wheeled-legged robot. In *2022 International Conference on Service Robotics (ICoSR)*, pages 31–37, 2022.